

Cyber Security Lab Report — Part 2

PROJECT 3: PHISHING EMAIL SIMULATION

Objective: Deploy a local Flask web application to simulate phishing landing pages and log employee interaction metrics for awareness training.

SOURCE CODE IMPLEMENTATION (PYCHARM STYLE)

```
from flask import Flask, request, render_template_string
import datetime

app = Flask(__name__)

LANDING_PAGE = """
<h2>Security Awareness Check</h2>
<form method="POST" action="/submit">
    <input type="text" name="email" placeholder="Email" required>
    <button type="submit">Verify</button>
</form>
"""

@app.route('/', methods=['GET'])
def index():
    user_ip = request.remote_addr
    timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    print(f"[{timestamp}] Link opened by IP: {user_ip}")
    return render_template_string(LANDING_PAGE)

@app.route('/submit', methods=['POST'])
def submit():
    email = request.form.get('email', 'Unknown')
    user_ip = request.remote_addr
    timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    print(f"[{timestamp}] Interaction recorded for: {email} from IP: {user_ip}")
    return "<h2>Notice: Simulated exercise. No data compromised.</h2>"

if __name__ == "__main__":
    app.run(port=5000, debug=False)
```

EXECUTION & SERVER LOG OUTPUT EXAMPLES

```
$ python3 phishing_sim.py
* Serving Flask app 'phishing_sim'
* Running on http://127.0.0.1:5000 (Press CTRL+C to quit)

--- SIMULATION EVENTS LOGGED ---
[2026-09-24 17:10:02] Link opened by IP: 127.0.0.1
127.0.0.1 - - [24/Sep/2026 17:10:02] "GET / HTTP/1.1" 200 -

[2026-09-24 17:10:18] Interaction recorded for: user@company.com from IP: 127.0.0.1
127.0.0.1 - - [24/Sep/2026 17:10:18] "POST /submit HTTP/1.1" 200 -
```

Cyber Security Lab Report — Part 2

PROJECT 4: SIMPLE VULNERABILITY SCANNER

Objective: Build a socket-based network port scanner to probe target IP addresses, detect open TCP ports, and grab service banners.

SOURCE CODE IMPLEMENTATION (PYCHARM STYLE)

```
import socket

COMMON_PORTS = {21: "FTP", 22: "SSH", 80: "HTTP", 443: "HTTPS", 8080: "HTTP-Proxy"}

def scan_target(target_host, ports):
    print(f"Scanning target: {target_host}\n" + "-"*40)

    for port in ports:
        service_name = COMMON_PORTS.get(port, "Unknown")
        try:
            sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            sock.settimeout(1.5)

            result = sock.connect_ex((target_host, port))
            if result == 0:
                try:
                    sock.send(b'HEAD / HTTP/1.0\r\n\r\n')
                    banner = sock.recv(1024).decode('utf-8', errors='ignore').strip()
                    summary = banner.split('\n')[0] if banner else "No banner returned"
                except Exception:
                    summary = "Banner retrieval failed"
                print(f"[OPEN] Port {port} ({service_name}) - Banner: {summary}")
            else:
                print(f"[CLOSED] Port {port} ({service_name})")
            sock.close()
        except Exception as e:
            print(f"Error scanning port {port}: {e}")

if __name__ == "__main__":
    scan_target("127.0.0.1", [21, 22, 80, 443, 8080])
```

EXECUTION & TEST OUTPUT EXAMPLES

```
$ python3 vuln_scanner.py

Scanning target: 127.0.0.1
-----
[CLOSED] Port 21 (FTP)
[OPEN]   Port 22 (SSH) - Banner: SSH-2.0-OpenSSH_8.9p1 Ubuntu-3ubuntu0.10
[OPEN]   Port 80 (HTTP) - Banner: HTTP/1.1 200 OK
[CLOSED] Port 443 (HTTPS)
[OPEN]   Port 8080 (HTTP-Proxy) - Banner: HTTP/1.0 400 Bad Request
```

KEY OBSERVATIONS

- Service banners reveal operating system and software version numbers, aiding vulnerability assessment against known CVE database records.
- Network administrators can obscure or disable banner responses to prevent unauthenticated information disclosure.